Отчёт по проекту «Ассистент сервисной службы на Graph RAG»

Репозиторий команды: https://github.com/sanyagataulin/llm-project.git

Участник: **Гатаулин Александр Денисович**

Модель: **qwen3:4b-instruct-2507-q4_K_M** (Ollama, температура 0), эмбеддинги: **bge-m3**.

Оглавление

1 Задача 3

2 Решение 3

2.1 Извлечение графа (extraction.py, FR-1) 4

2.2 Сборка графа (graph_builder.py, FR-2) 4

2.3 Сообщества и сводки (communities.py, FR-3) 4

2.4 Поиск по графу (search.py, FR-4, FR-5) 5

2.5 Инструменты и агент (tools.py, agent.py, FR-6 – FR-10) 5

3 Результаты 6

4 Где граф помогает и где уступает 8

4.1 Помогает: цепочка через несколько документов 8

4.2 Уступает: обзорные вопросы по редким фактам 8

5 Ошибки 9

6 Границы системы 10

7 Следующий шаг 10

8 Вклад участников 11

9 Использование ИИ-инструментов 11

1 Задача

«ПолярТех» обслуживает холодильное оборудование трёх линеек: витрины «Арктика», моноблоки «Бриз» и чиллеры «Гольфстрим». Заявку принимает диспетчер: ему называют модель и код ошибки. Чтобы отправить инженера с нужной деталью, требуется выяснить пять вещей: какой узел отказал, какая деталь стоит на этом месте в этой модели, не снята ли она с производства и чем заменена, есть ли она на ближайшем складе и сколько стоит. Первые три ответа разбросаны по 80 документам (руководства, сервисные бюллетени, регламенты, отчёты инженеров), остатки и цены живут только в учётной системе ERP.

Сейчас поиск по документам занимает у диспетчера около 12 минут, и примерно в каждой двенадцатой заявке заказывается деталь, снятая с производства: ERP такой заказ отклоняет, и замену приходится искать заново.

Готовый плоский RAG (чанки → эмбеддинги → ближайшие чанки) отвечает верно лишь в половине случаев. Характерный пример — вопрос «Какую деталь заказать при ошибке E-17 на AR-210?». В руководстве сказано, что E-17 означает отказ пускового реле и что в AR-210 это деталь P-1042; о замене P-1042 на P-1042B написано только в бюллетене, который по смыслу не похож на вопрос. Плоский RAG советует заказать «пусковое реле» — в ответе он называет узел вместо кода детали, потому что бюллетень с заменой в выборку не попадает. Правильный ответ — **P-1042B**, и он лежит на пути по графу знаний E-17 → пусковое реле → P-1042 → P-1042B.

2 Решение

Ассистент построен по схеме Graph RAG: документы превращаются в граф знаний (направленный `nx.DiGraph`), поиск отвечает по фактам графа и сводкам его сообществ, а агент объединяет поиск с инструментами ERP. Все обращения к модели идут через `LLMClient`: ответы кешируются на диске (`artifacts/llm_cache/`), температура равна 0, вызовы и токены считаются.

2.1 Извлечение графа (extraction.py, FR-1)

Функция `extract_chunk` вызывает модель с `json_schema=EXTRACTION_SCHEMA`. Системный промпт перечисляет 7 типов сущностей и 8 типов связей с явным направлением каждой связи и содержит разобранный пример (бюллетень о замене детали). Ответ разбирается через pydantic-модель; при ошибке разбора делается одна повторная попытка с уточняющей подсказкой, после второй неудачи возвращается пустой результат.

Всё, что не подтверждено текстом, отбрасывается и считается в поле `dropped`: код, которого нет в чанке (проверка `find_codes` после `normalize_code` — поэтому «BZ 210» из отчёта приводится к `BZ-210`), название не из словаря `vocabulary`, связь с отброшенным концом. Самостоятельно принятые решения: **если связь не проходит схему, проверяется развёрнутая** — модель часто путает направление `IS_A`; **для руководств `manual_<Model>.md` связь `HAS_PART` достраивается детерминированно** — строка «узел – деталь P-код» означает установку детали в модель из заголовка, а модель эту связь стабильно пропускает.

2.2 Сборка графа (graph_builder.py, FR-2)

`build_graph` склеивает результаты всех чанков в один граф. Узел — идентификатор вида `Part:P-1042` с атрибутами `type`, `name`, `sources`; ребро — `type`, `weight`, `sources`. Вес ребра равен числу **разных чанков**, где связь найдена: поэтому отказ, зафиксированный в 15 отчётах, имеет вес 15. Связи вне схемы (`Relation.matches_schema`) отбрасываются. Конфликт «`REPLACED_BY` в обе стороны» разрешается по весу: остаётся ребро с большим весом, при равенстве удаляются оба; число конфликтов пишется в `graph.graph["conflicts"]`.

2.3 Сообщества и сводки (communities.py, FR-3)

`detect_communities` разбивает граф алгоритмом Лувена с фиксированным `seed=7`; направление рёбер игнорируется, веса параллельных рёбер суммируются, сообщества нумеруются с нуля, каждый

узел входит ровно в одно. `summarize_community` пишет сводку до 150 слов одним вызовом модели. Принципиальное решение: **все числа считает код** — модель получает готовый список сущностей по типам, рёбра `FAILED` с весами «число отчётов» и перечень замен `REPLACED_BY`, а не считает их сама.

2.4 Поиск по графу (search.py, FR-4, FR-5)

`local_search` отвечает по конкретным сущностям. Опорные сущности извлекаются без модели: коды — через `find_codes`, названия — через `find_names` по словарю. Если код из вопроса отсутствует в графе или фактов не нашлось, сразу возвращается отказ без вызова модели. При двух и более опорных сущностях берутся рёбра на всех простых путях длиной до трёх связей (направление игнорируется), при одной — окрестность на глубину два. К фактам достраиваются цепочки `REPLACED_BY` от деталей на путях и связи `SUPPLIED_BY`, чтобы модель могла назвать актуальную деталь и поставщика. Модель получает пронумерованный список фактов вида `E-17 -INDICATES-> пусковое реле` объёмом до 8000 символов и обязана закончить ответ строкой «Ответ: …». Добавленная самостоятельно проверка: если при непустом списке фактов модель всё же отказалась, ей отправляется запрос перепроверить список — это снимает ложные отказы.

`global_search` отвечает на обзорные вопросы в два шага по сводкам сообществ. На шаге map модель по каждой сводке возвращает JSON по `MAP_SCHEMA`: частичный ответ и оценку полезности 0–10; нечитаемые ответы пропускаются. На шаге reduce частичные ответы с оценкой выше 0 объединяются одним вызовом; в промпте перед каждым частичным ответом перечислены сущности его сообщества, чтобы модель отбрасывала части про другие линейки. Если полезных частичных ответов нет — `refused=True`.

2.5 Инструменты и агент (tools.py, agent.py, FR-6 – FR-10)

`TOOL_SCHEMAS` описывает пять инструментов в формате function calling: `search_knowledge` (local_search), `summarize_topic` (global_search), `check_stock`, `get_price` и `create_service_order` (заказ в ERP). Каждая

функция сначала проверяет аргументы: склад из перечисления, номер детали вида `P-1042`/`P-1042B`, количество больше нуля. `ErpNotFound` и `ErpRejected` превращаются в словарь с `error` разным текстом, чтобы ответы «данных нет» и «остаток 0» оставались различимыми; `ErpUnavailable` пробрасывается агенту для повторов.

`run_agent` ведёт цикл «модель → вызов инструмента → наблюдение» до `max_steps=6` вызовов; каждый шаг записывается. Код агента (не модель) проверяет соблюдение правил: номер детали в аргументах должен встречаться в вопросе или результатах прошлых инструментов — иначе вызов отклоняется с подсказкой вызвать `search_knowledge`; количество в заказе должно быть названо в вопросе — иначе уточняющий вопрос; повторный вызов с теми же аргументами получает прежний результат из кеша шагов. На `ErpUnavailable` делается до трёх попыток. Если модель ответила отказом, ни разу не вызвав `search_knowledge` при наличии кода в вопросе, агент один раз подталкивает её подсказкой. Перед выдачей ответ сверяется с доказательствами: каждый код и каждое число с единицами «руб»/«шт» должны встречаться в вопросе или результатах инструментов; при расхождении модель один раз просится исправить ответ, после второго расхождения возвращается отказ.

Создание заказа отделено от ответа: при `confirm=False` агент останавливается и возвращает `needs_confirmation=True` с `pending_action` — решение о заказе всегда остаётся за человеком (см. раздел 6).

3 Результаты

Все режимы отвечают на одни и те же 24 открытых вопроса. Метрики посчитаны оценщиком проекта (`make eval-all`, `make eval-extraction`); в ERP обращается только агент, поэтому у остальных режимов точность на вопросах типа `tool` равна нулю.

Таблица 1 — Сравнение четырёх режимов на 24 вопросах (модель qwen3:4b-instruct-2507-q4_K_M)

Метрика	baseline	local	global	agent
Доля верных ответов	0,500	0,667	0,500	0,917
local (5 вопросов)	1,000	1,000	0,600	1,000
multihop (7 вопросов)	0,571	1,000	0,571	0,857
global (4 вопроса)	0,000	0,250	0,750	0,750
tool (5 вопросов)	0,000	0,000	0,000	1,000
refusal (3 вопроса)	1,000	1,000	0,667	1,000
Доля выдуманных фактов	0,042	0,000	0,000	0,000
Recall источников	0,856	1,000	0,875	1,000
Доля отклонённых вызовов	0,000	0,000	0,000	0,029
Вызовов модели на вопрос	1,0	1,04	9,88	5,71
Токенов на вопрос	1357	1022	6626	6254

Режим `local` полностью решает вопросы типов `local` и `multihop` и стоит дешевле плоского RAG (около одного вызова и 1022 токенов на вопрос): обход графа заменяет несколько чанков текста компактным списком фактов. Режим `global` лучше всех без агента отвечает на обзорные вопросы (0,75), но дорог — почти 10 вызовов модели на вопрос. Агент снимает ограничения обоих: 22 верных ответа из 24, включая все вопросы, требующие ERP.

Таблица 2 — Качество извлечения на эталоне из 16 документов (make eval-extraction)

Метрика	Сущности	Связи
Precision	1,000	1,000

Метрика	Сущности	Связи
Recall	1,000	0,981
Найдено / эталон	169 / 165	162 / 161

Отброшено проверками FR-1: 9 сущностей и связей. Индексация всего корпуса (`make index`): 92 чанка, **118 узлов**, **275 рёбер**, **9 сообществ**, отброшено 87 фактов, конфликтов `REPLACED_BY` не зафиксировано. Стоимость: 101 вызов модели, 99 289 токенов промптов и 47 248 токенов ответов (~146,5 тыс. токенов, т.е. около 1600 токенов на чанк); за счёт кеша повторные прогоны занимают секунды.

4 Где граф помогает и где уступает

4.1 Помогает: цепочка через несколько документов

Вопрос «Какую деталь заказать при ошибке E-17 на AR-210?» (тип `multihop`). Плоский RAG выбирает руководство и похожие отчёты и советует заказать «пусковое реле» — название узла вместо кода детали: бюллетень с заменой на P-1042B не похож на вопрос и в выборку не попадает. Режим `local` находит опорные сущности `E-17` и `AR-210`, берёт факты на путях между ними и достраивает цепочку `P-1042 -REPLACED_BY-> P-1042B`, поэтому отвечает **P-1042B** с источником `bulletin_SB-2025-03.md`. На всех семи вопросах `multihop` `local` верен, агент — в шести из семи.

4.2 Уступает: обзорные вопросы по редким фактам

Вопрос «Какие детали из руководств линейки „Бриз“ сняты с производства?» (dev-16). `local` проигрывает принципиально: обход от опорных сущностей не даёт полного перечисления, поэтому на типе `global` у него точность 0,25. Но и `global`, и агент на этом вопросе ошибаются: сводки сообществ — это тексты до 150 слов, редкие замены в них теряются, и шаг reduce собрал неполный ответ, назвав P-1046 и P-1046B снятыми деталями «Бриза» (P-1046 относится к «Арктике»). Зеркальная слабость `global` — узкие вопросы (dev-01, dev-02): конкретный регламент или узел не попал в сводку, score на map равен 0, и режим отказывается там, где `local` отвечает точно.

5 Ошибки

Ниже — разобранные ошибки системы с указанием этапа, на котором каждая возникла.

1. Ложный отказ при полных фактах (dev-10, агент, этап: модель составляет ответ по фактам в local_search). На вопрос «По какому регламенту устраняют отказ узла „вентилятор испарителя“?» инструмент `search_knowledge` вернул правильный список фактов — в нём есть `E-18 -FIXED_BY-> SP-108`. Но модель, читая список, объявила, что связей с регламентами нет, и ответила отказом. Данные и поиск сработали верно, ошиблась генерация по фактам — повторная проверка списка (recheck) этот случай не спасла.

2. Неполный агрегат на обзорном вопросе (dev-16, агент и global, этап: reduce по сводкам). Частичные ответы map содержали нужные сведения, но reduce объединил их неверно: назвал P-1046/P-1046B снятыми деталями «Бриза», потеряв полный перечень и приписав деталь чужой линейке. Первопричина — на шаге построения сводок: текст в 150 слов не сохраняет редкие факты о заменах.

3. Отказ на узком вопросе в режиме global (dev-01, dev-02, этап: map по сводкам). Вопросы «По какому регламенту устраняют ошибку E-17?» и «На отказ какого узла указывает E-22?» требуют единственного факта. В сводках сообщества он отсутствует, все частичные ответы получают score 0, и режим честно отказывается — ошибка не модели, а покрытия: сводки проецируют граф с потерей точных связей.

4. Потеря и подмена кода детали (dev-06, dev-07, global, этап: reduce). На dev-06 `global` отвечает «пусковое реле» — сводка передаёт вид узла, но не цепочку `REPLACED_BY` до кода детали. На dev-07 сводка того же сообщества хранит цепочку `P-3062 → P-3062B → P-3062C`, а связь GS-510 с деталью `P-3162` в ней теряется — reduce переносит чужую цепочку на вопрос и выдаёт `P-3062C` вместо верного `P-3162`.

6 Границы системы

Ассистент не выполняет действий с реальными последствиями без человека. `create_service_order` — единственный инструмент, меняющий состояние ERP, и он срабатывает только при `confirm=True`. При `confirm=False` агент останавливается и возвращает `needs_confirmation=True` и `pending_action` с инструментом и аргументами: диспетчер видит «создать заказ на деталь P-1042B в количестве 2 шт. для BZ-120 на складе Москва» и подтверждает его отдельно.

Подтверждение подстраховано проверками кода: количество в заказе должно быть явно названо в вопросе — иначе агент задаёт уточняющий вопрос, а не выдумывает число; номер детали в аргументах должен встречаться в вопросе или результатах инструментов — подставленный моделью «из головы» код отклоняется до обращения к ERP; каждый вызов валидируется самой функцией инструмента (склад из перечисления, форматы номера детали `P-1042` и кода модели `AR-210`, количество > 0). Перед выдачей финального ответа коды, цены и остатки сверяются с результатами инструментов — на прогоне это дало **нулевую долю выдуманных фактов** во всех трёх режимах Graph RAG.

7 Следующий шаг

Идея: при построении сводки сообщества дополнительно передавать модели **полный перечень рёбер `REPLACED_BY` и `FIXED_BY`** (старая деталь → новая, ошибка → регламент) и требовать упомянуть их в сводке отдельным предложением. Сейчас эти редкие, но критичные связи теряются в тексте до 150 слов — это первопричина ошибок раздела 5 (dev-16, dev-01/02, dev-06/07).

Проверка воспроизводима: перестроить сводки (включая кеш), прогнать `make eval MODE=global` и `make eval MODE=agent` на тех же 24 вопросах и сравнить точность на типах `global` (сейчас 0,75) и `multihop` у агента (0,857) — ожидаемый эффект: исправление dev-16 и dev-10 без роста числа вызовов модели.

8 Вклад участников

Проект выполнен одним участником (Гатаулином Александром Денисовичем): шесть модулей — `extraction.py`, `graph_builder.py`, `communities.py`, `search.py`, `tools.py`, `agent.py` — а также прогоны оценки и подготовка настоящего отчёта.

9 Использование ИИ-инструментов

Использовался агент **Devin**: он построил план решения и написал код модулей, а также сформировал настоящий отчёт по артефактам прогонов. Точечные правки в код вносились вручную, важные решения по реализации принимались вручную, тестирование и проверка результатов выполнялись вручную.